Evidencia de Aprendizaje 2. Diseño Sistema de Información – Iteración 2

Gestión de usuarios y gestión de celdas

Juan David Henao Cárdenas
Jorge Eliécer Uribe Pulgarín
Santiago Andrés Valencia Duque

Institución Universitaria Digital de Antioquia IU Digital
Ingeniería de Software y Datos
Ingeniería de Software
Profesor: Reinaldo Rojas López
2026

Grupo 18

Repositorio GitHub: https://github.com/andresvd95/IngSoftIUDig.git

Introducción

Este documento presenta los entregables de la Iteración 2 del sistema de información del parqueadero «Autos Colombia»: la gestión de usuarios (clientes y sus vehículos) y la gestión de celdas. Sobre la base de la Iteración 1 (entrada y salida de vehículos) se especifican los requerimientos funcionales y no funcionales, los casos de uso, las historias de usuario, los mockups, los diagramas de secuencia de ambas iteraciones, el diagrama de componentes y el plan de pruebas resumido.

La implementación correspondiente está en el repositorio de GitHub: backend en Spring Boot (Java 21, MySQL 8 y Flyway) y frontend en Angular, ambos integrados mediante una API REST.

Contenido de los entregables

1. Requerimientos funcionales y no funcionales
2. Diagrama de casos de uso
3. Historias de usuario
4. Mockups
5. Diagramas de secuencia (iteraciones 1 y 2)
6. Diagrama de componentes
7. Plan de pruebas resumido
8. Repositorio y video

1. Requerimientos funcionales y no funcionales

Requerimientos funcionales

ID	Nombre	Descripción
RF-04	Registrar cliente	El sistema debe permitir registrar un cliente con tipo y número de identificación, nombre, correo y teléfono.
RF-05	Consultar y buscar clientes	El sistema debe listar los clientes y permitir buscarlos por nombre o identificación.
RF-06	Editar y eliminar cliente	El sistema debe permitir actualizar los datos de un cliente y eliminarlo cuando ninguno de sus vehículos esté dentro del parqueadero.
RF-07	Registrar vehículo	El sistema debe asociar vehículos (placa, tipo, marca, modelo, color) a un cliente existente.
RF-08	Crear celda	El sistema debe permitir crear celdas con número único y tipo de vehículo admitido.
RF-09	Editar y eliminar celda	El sistema debe permitir modificar o eliminar una celda siempre que no esté ocupada.
RF-10	Consultar estado de celdas	El sistema debe mostrar cada celda con su tipo y estado (disponible u ocupada).

Requerimientos no funcionales

ID	Atributo	Descripción
RNF-04	Integridad	No pueden existir dos clientes con el mismo tipo y número de identificación, dos vehículos con la misma placa ni dos celdas con el mismo número (validación en servicio e índices únicos).
RNF-05	Concurrencia	Las operaciones sobre celdas y estancias usan bloqueo pesimista (SELECT … FOR UPDATE) para evitar condiciones de carrera.
RNF-06	Usabilidad	Los errores de validación y de reglas de negocio se muestran en la interfaz con un mensaje claro (HTTP 400, 404 y 409).
RNF-07	Rendimiento	Los listados responden de forma paginada y en menos de 2 segundos con hasta 1.000 registros.
RNF-08	Mantenibilidad	Arquitectura por capas (controller, service, persistence), migraciones Flyway y pruebas automatizadas.

Los requerimientos RF-01 a RF-03 y RNF-01 a RNF-03 corresponden a la Iteración 1 (entrada y salida de vehículos).

2. Diagrama de casos de uso

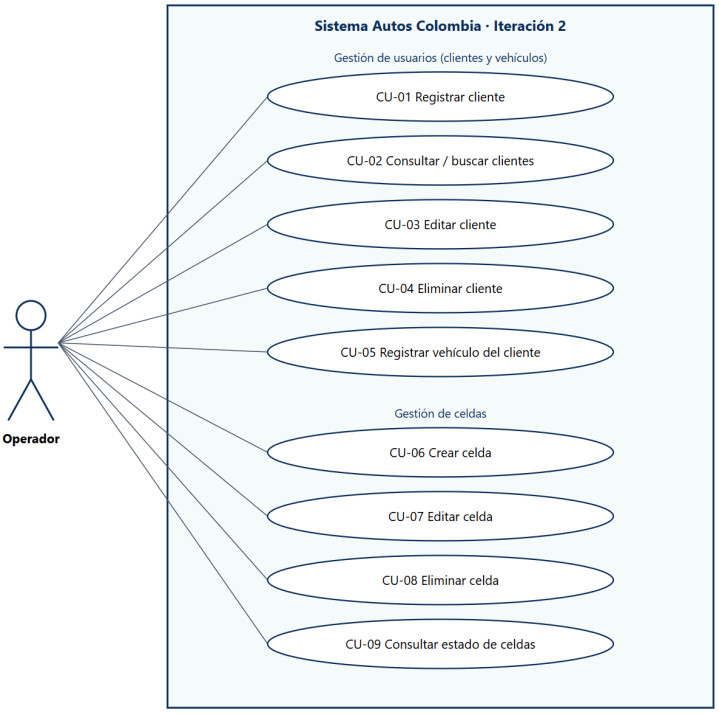


Figura 1. Casos de uso de gestión de usuarios y celdas (actor: Operador).

3. Historias de usuario

HU-04 — Registrar cliente
Como operador, **quiero** registrar los datos de un cliente, **para** tener su información disponible al ingresar sus vehículos.

1. Tipo, número de identificación y nombre son obligatorios; el correo debe tener formato válido.
2. Si ya existe un cliente con el mismo tipo y número se rechaza (HTTP 409).
3. Al guardar, el cliente aparece de inmediato en el listado y persiste al recargar.

HU-05 — Consultar, editar y eliminar clientes
Como operador, **quiero** buscar, modificar y eliminar clientes, **para** mantener actualizada la base de usuarios.

1. El listado se filtra por nombre o identificación.
2. La edición valida los mismos campos y rechaza duplicados (409).
3. No se puede eliminar un cliente con un vehículo dentro del parqueadero (409); si no existe, se informa (404).

HU-06 — Registrar vehículo del cliente
Como operador, **quiero** asociar un vehículo a un cliente, **para** poder registrar su ingreso por placa.

1. Placa, tipo de vehículo y cliente son obligatorios.
2. Una placa ya registrada se rechaza (409); el cliente o tipo inexistente devuelve 404.
3. El vehículo queda disponible en la vista de ingreso.

HU-07 — Crear celda
Como operador, **quiero** crear una celda indicando número y tipo, **para** ampliar la capacidad del parqueadero.

1. Número y tipo de vehículo son obligatorios.
2. Un número de celda duplicado se rechaza (409).
3. La nueva celda queda Disponible y visible en el estado de celdas.

HU-08 — Editar y eliminar celda
Como operador, **quiero** modificar o eliminar una celda, **para** reflejar cambios en la distribución física.

1. Una celda ocupada no se puede editar ni eliminar (409).
2. El nuevo número no puede coincidir con el de otra celda (409).
3. Al eliminar, la celda desaparece del listado.

HU-09 — Consultar estado de celdas
Como operador, **quiero** ver todas las celdas con su estado, **para** saber dónde puedo ubicar un vehículo.

1. Cada celda muestra número, tipo y estado (Disponible/Ocupada).
2. El estado se lee del backend y se actualiza tras cada ingreso o salida.

4. Mockups

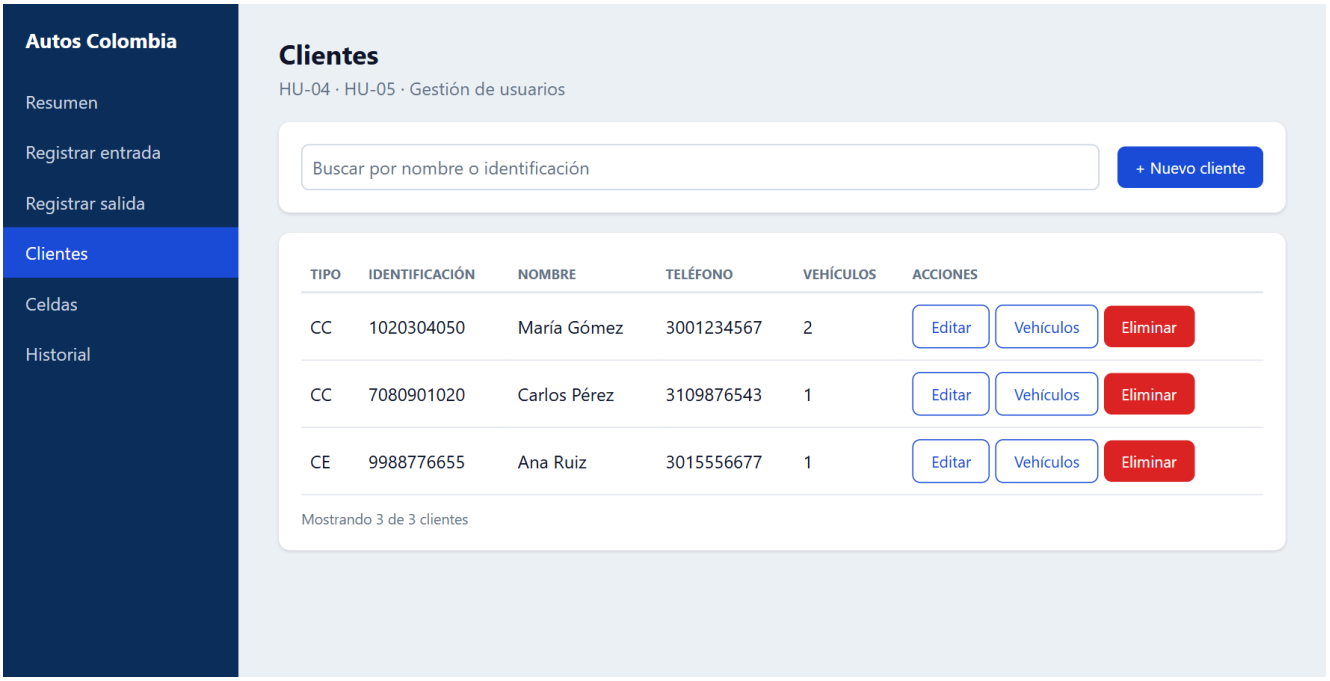


Figura 2. Listado y búsqueda de clientes (HU-04, HU-05).

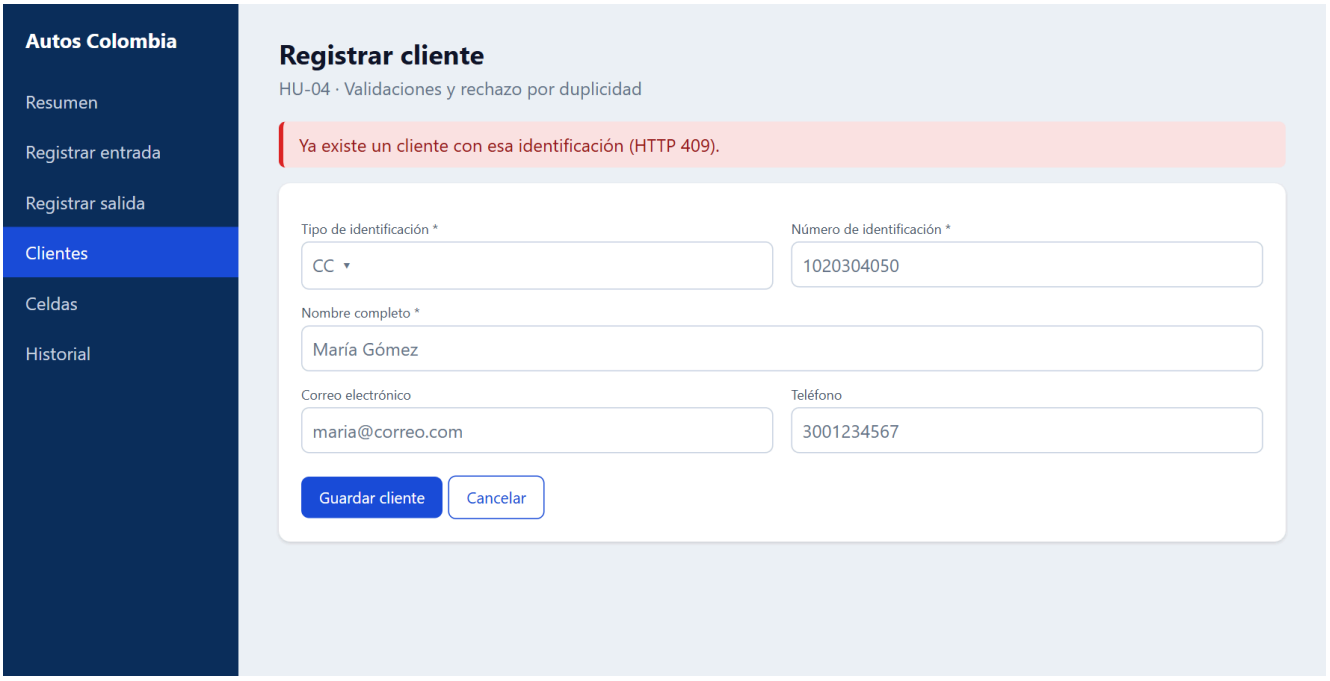


Figura 3. Formulario de cliente con rechazo por duplicidad (HU-04).

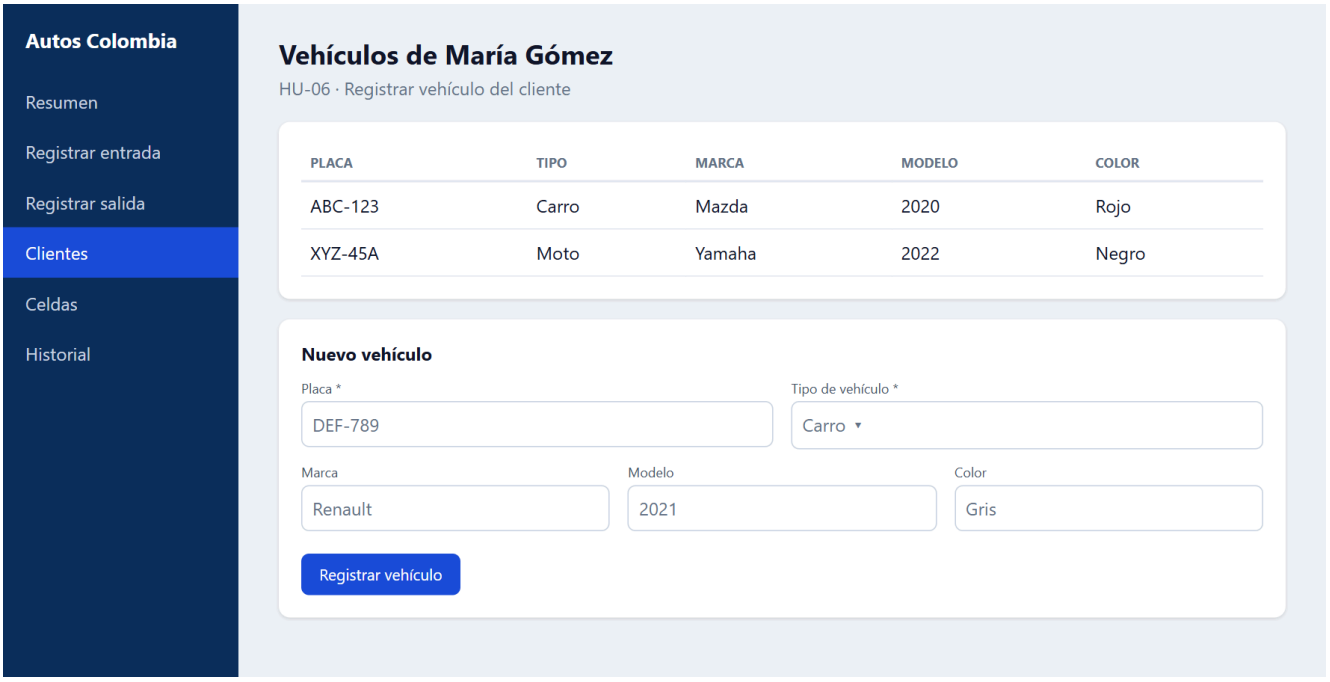


Figura 4. Registro de vehículos de un cliente (HU-06).



Figura 5. Gestión de celdas (HU-07, HU-08, HU-09).

5. Diagramas de secuencia

Iteración 1

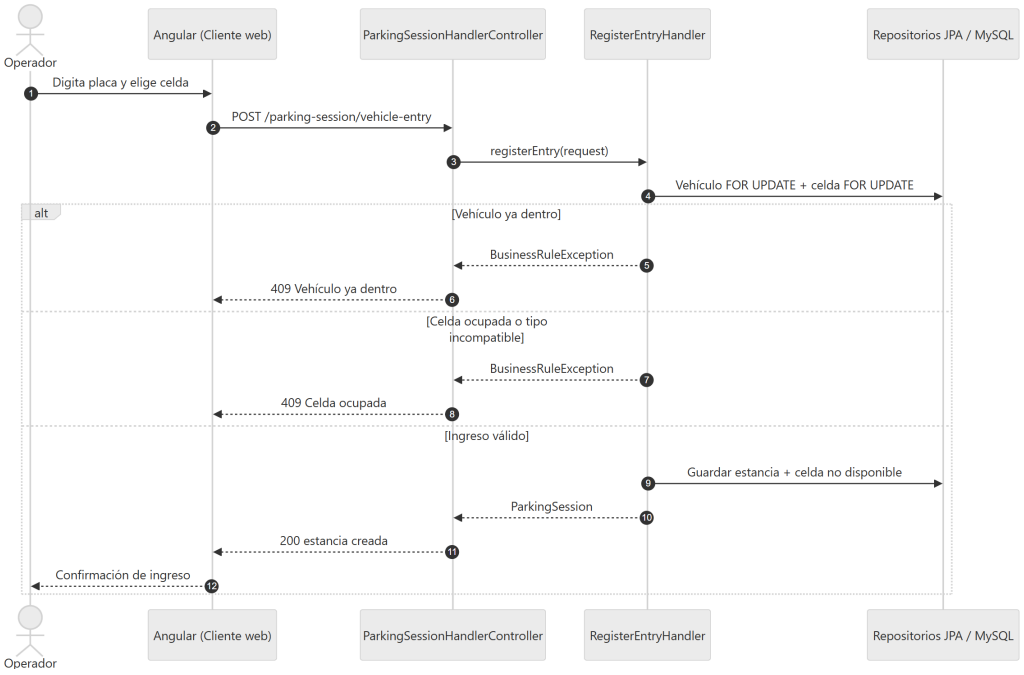


Figura 6. Registrar entrada de vehículo.

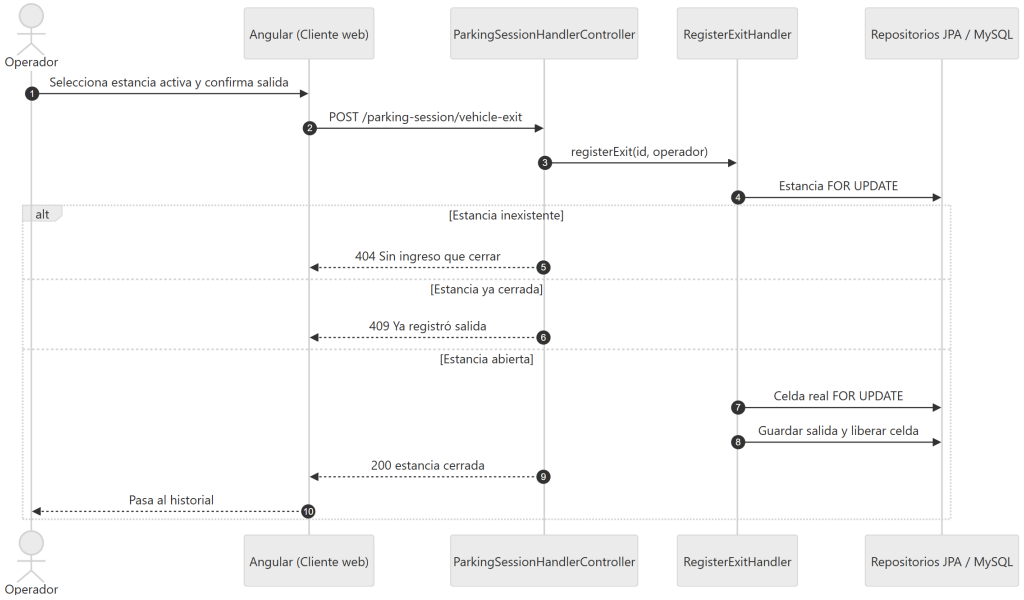


Figura 7. Registrar salida de vehículo.

Iteración 2

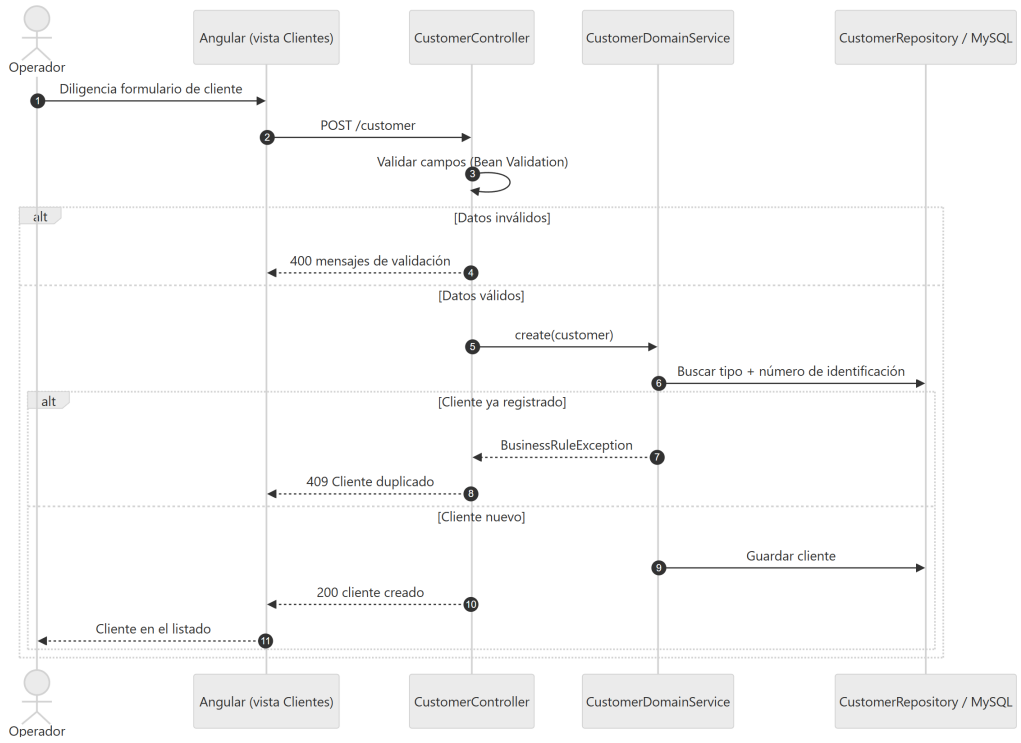


Figura 8. Registrar cliente.

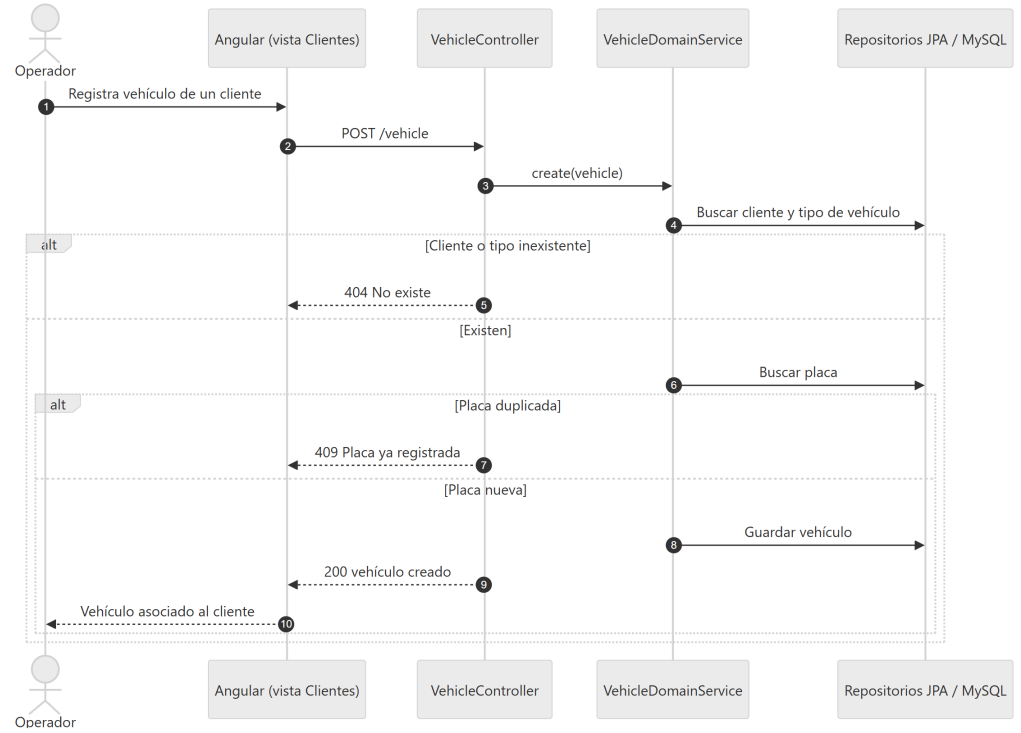


Figura 9. Registrar vehículo de un cliente.

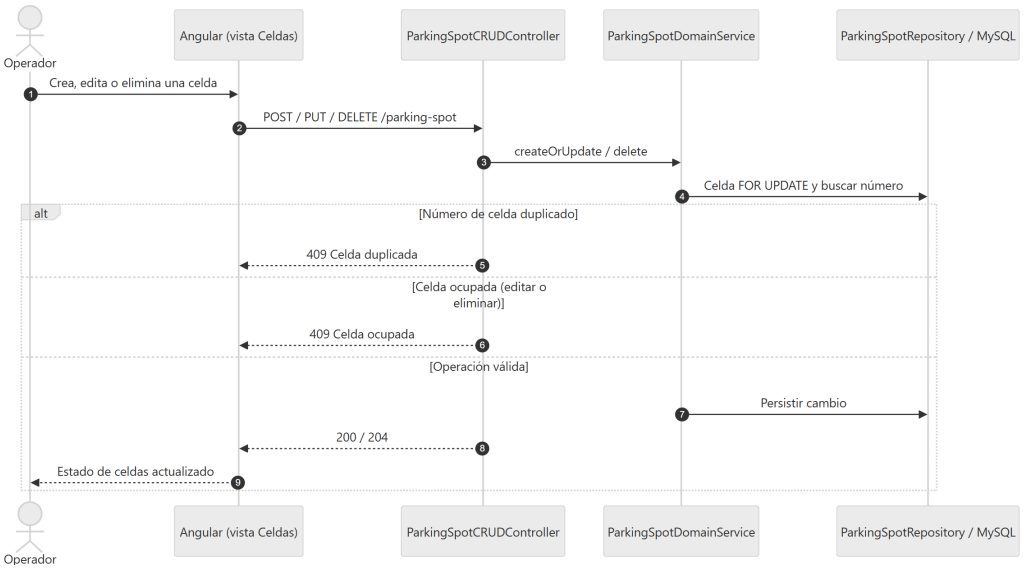


Figura 10. Crear, editar o eliminar celda.

6. Diagrama de componentes

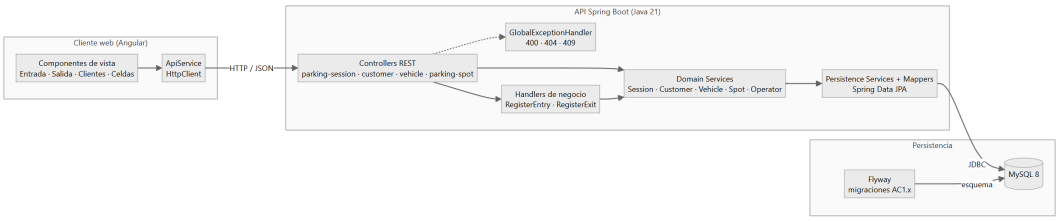


Figura 11. Componentes del sistema (iteraciones 1 y 2).

7. Plan de pruebas resumido

ID	HU	Caso de prueba	Datos de entrada	Resultado esperado	Resultado obtenido	Estado
PP-01	HU-04	Registrar cliente válido	CC 1020304050, María Gómez, correo y teléfono	200 y el cliente se puede consultar	ManagementApiTest.createsCustomerAndReadsItBack	Aprobada
PP-02	HU-04	Cliente con datos inválidos	Nombre vacío / correo mal formado	400 con mensajes de validación	ManagementApiTest.rejectsInvalidCustomerWith400	Aprobada
PP-03	HU-04	Cliente duplicado	Mismo tipo y número de identificación; mismo número con otro tipo	409 al duplicar; con otro tipo se acepta	ManagementApiTest.rejectsDuplicateCustomerIdentificationWith409ButAllowsOtherType	Aprobada
PP-04	HU-05	Buscar, editar cliente	Búsqueda por nombre; edición propia, con choque y con id inexistente	Filtra el listado; 200 / 409 / 404	ManagementApiTest.createsCustomerAndReadsItBack · updatesCustomerAndRejectsCollisionAndMissing	Aprobada
PP-05	HU-05	Eliminar cliente	Cliente sin estancias; luego repetir	204 y después 404	ManagementApiTest.deletesCustomerWith204AndThen404	Aprobada
PP-06	HU-05	Eliminar cliente con vehículo dentro	Cliente con estancia abierta	409 y el cliente se conserva	ManagementApiTest.blocksCustomerDeletionWhenAVehicleIsInside	Aprobada
PP-07	HU-06	Registrar vehículo y listar por cliente	Placa en minúsculas, tipo y cliente válidos	200, placa normalizada a mayúsculas, aparece en el cliente	ManagementApiTest.createsVehicleNormalizesPlateAndListsByCustomer	Aprobada
PP-08	HU-06	Placa duplicada	Placa ya registrada	409	ManagementApiTest.rejectsDuplicatePlateWith409	Aprobada
PP-09	HU-06	Vehículo con cliente o tipo inexistente / cuerpo inválido	customerId o vehicleTypeId inexistente	404; 400 si el cuerpo es inválido	ManagementApiTest.rejectsVehicleWithMissingCustomerOrTypeOrInvalidBody	Aprobada
PP-10	HU-07	Crear, editar y eliminar celda	Número A-12, tipo Carro	Celda creada disponible; edición y borrado correctos	ManagementApiTest.createsEditsAndDeletesSpot	Aprobada
PP-11	HU-07	Celda inválida o duplicada	Sin número / sin tipo; número repetido	400; 409	ManagementApiTest.rejectsInvalidSpotWith400AndDuplicateWith409	Aprobada
PP-12	HU-08	Editar celda con choque o inexistente	Número de otra celda; id inexistente	409; 404	ManagementApiTest.rejectsSpotEditCollisionAndMissing	Aprobada

PP-13	HU-08	Editar o eliminar celda ocupada	Celda con estancia abierta	409 y el flujo de ingreso/salida sigue funcionando	ManagementApiTest.rejectsEditAndDeletionOfOccupiedSpotAndFlowStillWorks	Aprobada
PP-14	HU-09	Estado de celdas tras ingreso y salida	Ingreso y salida de un vehículo	La celda pasa a Ocupada y vuelve a Disponible	ParkingSessionFlowTest (7 pruebas)	Aprobada

Las pruebas automatizadas están en `autos-colombia/src/test` y se ejecutan con `./mvnw test`.

8. Repositorio y video

Repositorio GitHub: https://github.com/andresvd95/IngSoftIUDig.git

Video en YouTube: PENDIENTE: pegar aquí la URL del video de YouTube

Conclusiones

La Iteración 2 completa el ciclo de datos maestros del parqueadero: ahora es posible registrar clientes y vehículos y administrar las celdas antes de operar entradas y salidas. Las reglas de integridad (duplicados, celdas ocupadas, clientes con estancia abierta) se aplican en el backend y se comunican al operador con mensajes claros.

La arquitectura por capas, el bloqueo pesimista y los índices únicos mantienen la consistencia ante operaciones simultáneas, y las pruebas automatizadas documentan el comportamiento esperado. Como trabajo futuro quedan la gestión de operadores, los pagos y las mensualidades de la siguiente iteración.